

Lista de Exercícios 03

Lista de Exercícios — Estruturas de Dados

Conteúdo: Árvores Binárias de Busca

01. Em uma ABB:

- a) Valores menores ficam à direita
- b) Valores maiores ficam à esquerda
- c) Valores menores ficam à esquerda e maiores à direita
- d) Não existe ordenação

02. Qual percurso em uma ABB retorna os valores ordenados?

- a) Pré-ordem
- b) Pós-ordem
- c) Em ordem
- d) Largura

03. A complexidade média de busca em uma ABB balanceada é:

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n^2)$
- d) $O(n!)$

04. Quando uma ABB se torna degenerada:

- a) Todos os nós possuem dois filhos
- b) Ela se comporta como uma lista encadeada
- c) Não existem folhas
- d) Todos os valores são iguais

05. Explique como funciona a inserção em uma ABB.

Resposta:

A inserção em uma ABB funciona do seguinte modo: tendo um valor que irá ser inserido, ele passa por uma comparação com o nó atual, seguindo as seguintes condições: se o valor inserido for menor que o nó comparado, ele vai para a subárvore da esquerda; caso seja maior, ele vai para a subárvore da direita. Esse processo se repete até encontrar um local vazio, onde o novo dado será inserido.

06. Explique como funciona a busca em uma ABB.

Resposta:

A busca em uma ABB funciona do seguinte modo: tendo o endereço da raiz e o valor que será buscado, ele passa pelas seguintes condições. Se o valor procurado for igual ao valor da raiz, a busca é encerrada e o resultado é retornado. Caso contrário, se o valor for maior que o nó atual, a busca continua pela subárvore da direita; se for menor, continua pela subárvore da esquerda. Esse processo se repete até encontrar o valor desejado ou chegar a um nó nulo, indicando que o elemento não pertence à árvore.

07. Explique os três casos de remoção em ABB:

- remoção de folha:

Funciona do seguinte modo: O valor excluído é localizado e depois removido sem precisar passar por uma reorganização da ABB.

- remoção de nó com um filho

Mas já a remoção com nó que possui um filho, o elemento eliminado é localizado, e como ele possui apenas um filho, ele apenas remove o nó e o filho herda a posição do pai, mantendo a estrutura da ABB.

- remoção de nó com dois filhos

Nesse caso, o elemento eliminado é localizado e, para assumir o posto do pai, olhamos para a subárvore da direita. Vamos caminhando por ela até achar o menor valor para assumir o posto do pai removido e estruturando aquela subárvore.

08. Qual a principal vantagem de uma ABB em relação a uma árvore binária comum?

A vantagem que a ABB tem em comparação com a árvore binária comum é que ela possui uma regra estrita onde o menor vai para a esquerda e o maior para a direita. Com isso, não é preciso que o dado buscado, removido ou inserido passe por toda a estrutura da árvore. Isso acontece pois nesse caso existem dois tipos de subárvore, a da esquerda e a da direita: se o valor for menor, a busca vai ser na subárvore da esquerda, caso contrário vai ser na subárvore da direita.

09. Explique o problema do desbalanceamento em ABB.

O desbalanceamento da ABB ocorre quando a árvore possui uma forma linear, ou seja, sendo ela para a esquerda ou para a direita. A árvore pode possuir apenas filhos na direita (todos os nós na direita) e o mesmo vale para a esquerda, tornando a árvore similar a uma estrutura de lista encadeada. O problema disso é que se perde a vantagem da ABB, pois as operações passam a ter que percorrer quase todos os nós, ficando muito mais lentas.

IMPLEMENTAÇÃO DE ABB

10. Implemente a função de inserção em uma ABB usando recursão.
11. Implemente a função de busca em uma ABB.
12. Implemente o percurso em ordem em uma ABB.
13. Implemente a remoção de um nó folha em uma ABB.
14. Implemente a remoção de um nó com apenas um filho.
15. Implemente a remoção de um nó com dois filhos utilizando sucessor em ordem.
16. Implemente uma função que encontre:
 - maior valor
 - menor valorem uma ABB.
17. Crie uma função que verifique se uma árvore é realmente uma ABB válida.
18. Implemente uma função que conte quantos níveis existem na ABB.
19. Crie um programa completo de ABB com menu contendo:
 - Inserir
 - Buscar
 - Remover
 - Imprimir em ordem
 - Mostrar altura
 - Encerrar

DESAFIOS (Nível Avançado)

20. Implemente uma ABB sem utilizar recursão.
21. Implemente uma árvore binária utilizando alocação dinâmica.
22. Crie uma função que transforme uma árvore binária em espelho.
23. Implemente uma função que verifique se uma árvore é balanceada.

24. Pesquise e explique:

- O que é AVL?

Resposta: A AVL vai ser uma árvore binária de busca que tem como principal característica se organizar sozinha, ou seja, ela é auto-balanceada. Ela passa por uma verificação toda vez que você coloca ou tira um nó para ver se a altura de um lado não está muito maior que a do outro. Se a diferença de altura entre a esquerda e a direita passar de 1, ela faz um processo chamado rotação para arrumar os nós e deixar a árvore reta de novo, o que faz as buscas continuarem muito rápidas.

- O que é árvores Rubro-Negra?

Resposta: É um outro tipo de árvore binária de busca que também se balanceia sozinha, mas em vez de ficar calculando a altura certinha dos nós, ela usa um esquema de cores onde cada nó vai ser pintado de vermelho (rubro) ou preto (negra). Ela tem umas regras fixas para essas cores, tipo a raiz tem que ser preta e um nó vermelho não pode ter filho vermelho. Isso serve para que nenhum caminho até as folhas fique muito maior que o outro, mantendo os dados equilibrados.

- O que são B-Tree?

Resposta: As árvores B já não são árvores binárias, porque elas guardam mais de um dado dentro do mesmo nó e também podem ter muito mais de dois filhos saindo de um pai só. Elas são muito utilizadas para mexer com grandes volumes de dados que ficam salvos no disco ou em sistemas de bancos de dados modernos, pois como os nós guardam muita informação junta, a árvore fica com poucos níveis de altura e o sistema consegue achar o que quer de forma muito mais rápida.